bpmn.pic

`bpmn.pic` is a collection of DPIC macros for generating BPMN 2.0 diagrams (as images) for use in various kinds of documents and presentations. Although the library focuses on supporting SVG as the primary output format, most of the elements (except pools and lanes) should also work with other DPIC formats.

Currently supported BPMN element types include:

- **Events**: start, non-interrupting, catching, boundary, throwing, end;
- **Tasks**: none, receive, send, manual, service, user, script, rule;
- **Sub-Processes**: collapsed, expanded;
- **Gateways**: none, exclusive, parallel, inclusive, event-based (exclusive, parallel);
- **Flows**: sequence flow, default condition flow, conditional flow, message flow;
- **Pools**: horizontal pools and lanes, horizontal black box pools.

The goal of `bpmn.pic` is to provide just the visual layer of BPMN 2.0. If you need full support for BPMN 2.0-compatible XML, please try other tools (like bpmn.io) instead.

Installation

Download `bpmn.pic` from GitLab:

```
$ wget https://gitlab.com/pkierat/bpmn-pic/-/raw/master/bpmn.pic
```

The file is self-contained, and has no external run-time dependencies apart from DPIC itself.

Building from source

Buiding `bpmn.pic` requires the following tools to be available on the `PATH`:

- Binutils
- Make
- M4 Macro Processor

Building `README.pdf` requires in addition:

- DPIC
- Asciidoctor PDF
- Asciidoctor Diagram

Procedure:

1. Clone the repository:

```
$ git clone git@gitlab.com:pkierat/bpmn-pic.git ./bpmn-pic
$ cd bpmn-pic
```

2. Build `bpmn.pic`:

```
$ make -C src bpmn.pic
```

3. (Optional) Build `README.pdf`:

```
$ make README.pdf
```

Usage

1. Include the `bpmn.pic` file in your PIC diagram using the `copy` statement:

```
.PS
copy "bpmn.pic"

Start(none); SequenceFlow(); Task("Task"); SequenceFlow(); End(none)
.PE
```

2. Generate the SVG file:

```
$ dpic -v diagram.pic > diagram.svg
```

Macros

Events

```
Start(type [, attributes ])
StartNonInterrupting(type [, attributes])
Boundary(type [, attributes])
BoundaryNonInterrupting(type [, attributes])
Catching(type [, attributes])
Throwing(type [, attributes])
End(type [, attributes])
```

Gateways

```
Gateway(type [, attributes ])
```

Activities

```
Task(text [, type [, attributes ] ])
SubProcess(text [, type [, attributes ] ])
```

Flows

```
SequenceFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
DefaultFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
ConditionalFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
MessageFlow([ linespec_1 [, linespec_2 [, ⋯ ] ] ])
```

Pools and lanes

```
HorizontalPool(width, height, lanes [, attributes ])
HorizontalBlackBoxPool(width, height [, text [, attributes ] ])
HorizontalLane(height, process [, attributes ])
Pool(width, height, lanes [, attributes ])
Lane(height, process [, attributes ])
```

Other

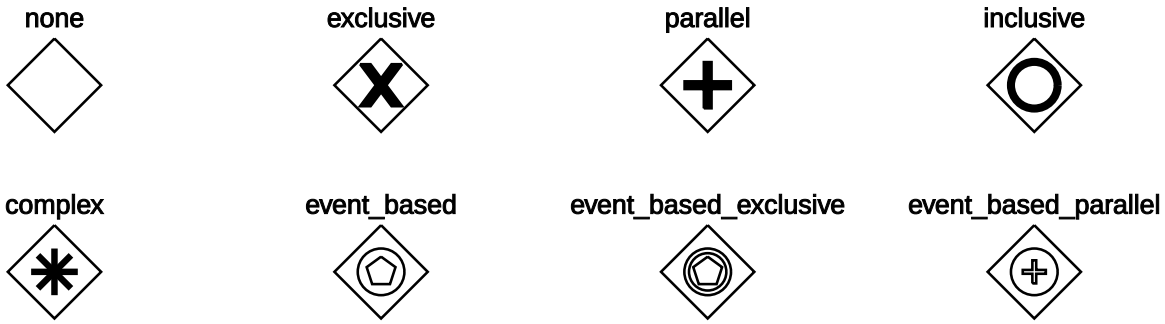
```
Text(text, position [, rotation ])
```

Elements

Events

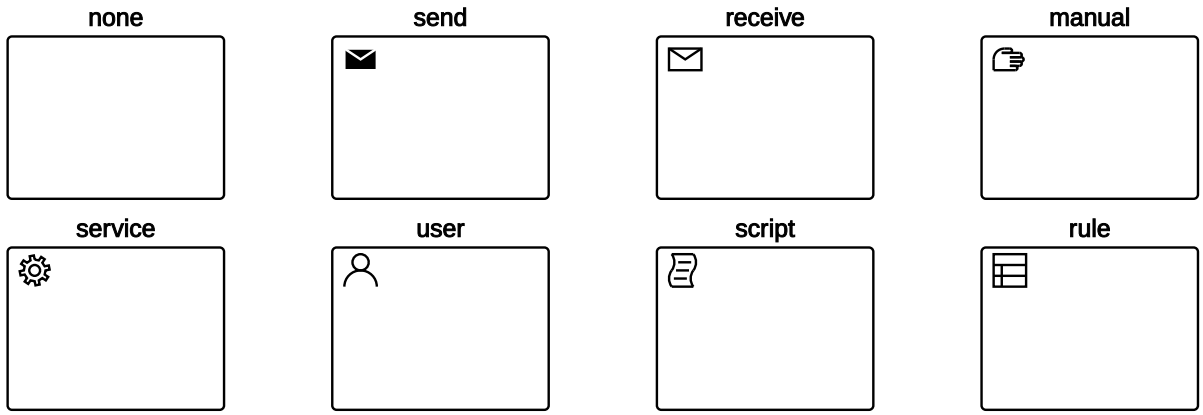
Type	Start	Start non-interrupting	Catching	Boundary non-interrupting	Throwing	End
none						
message						
timer						
signal						
multiple						
conditional						
link						
escalation						
error						
cancel						
parallel						
compensation						
terminate						

Gateways

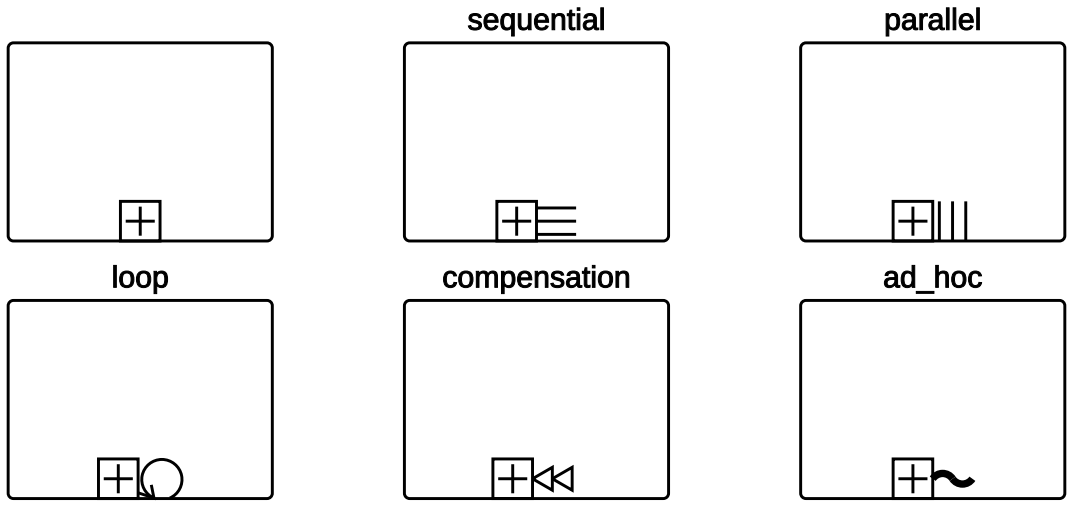


Activities

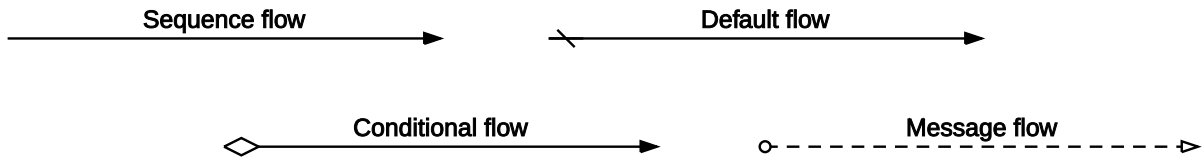
Tasks



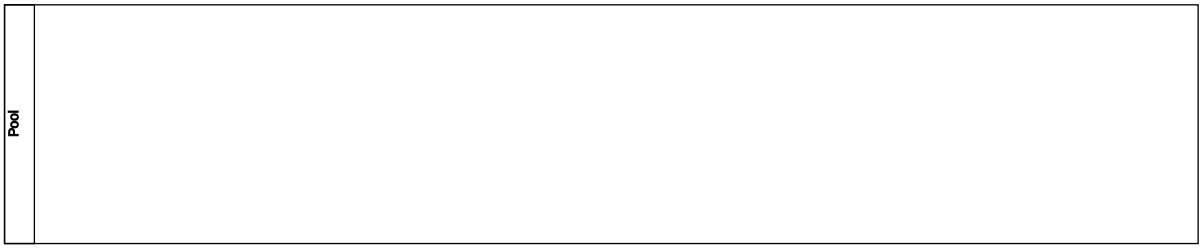
Sub-processes



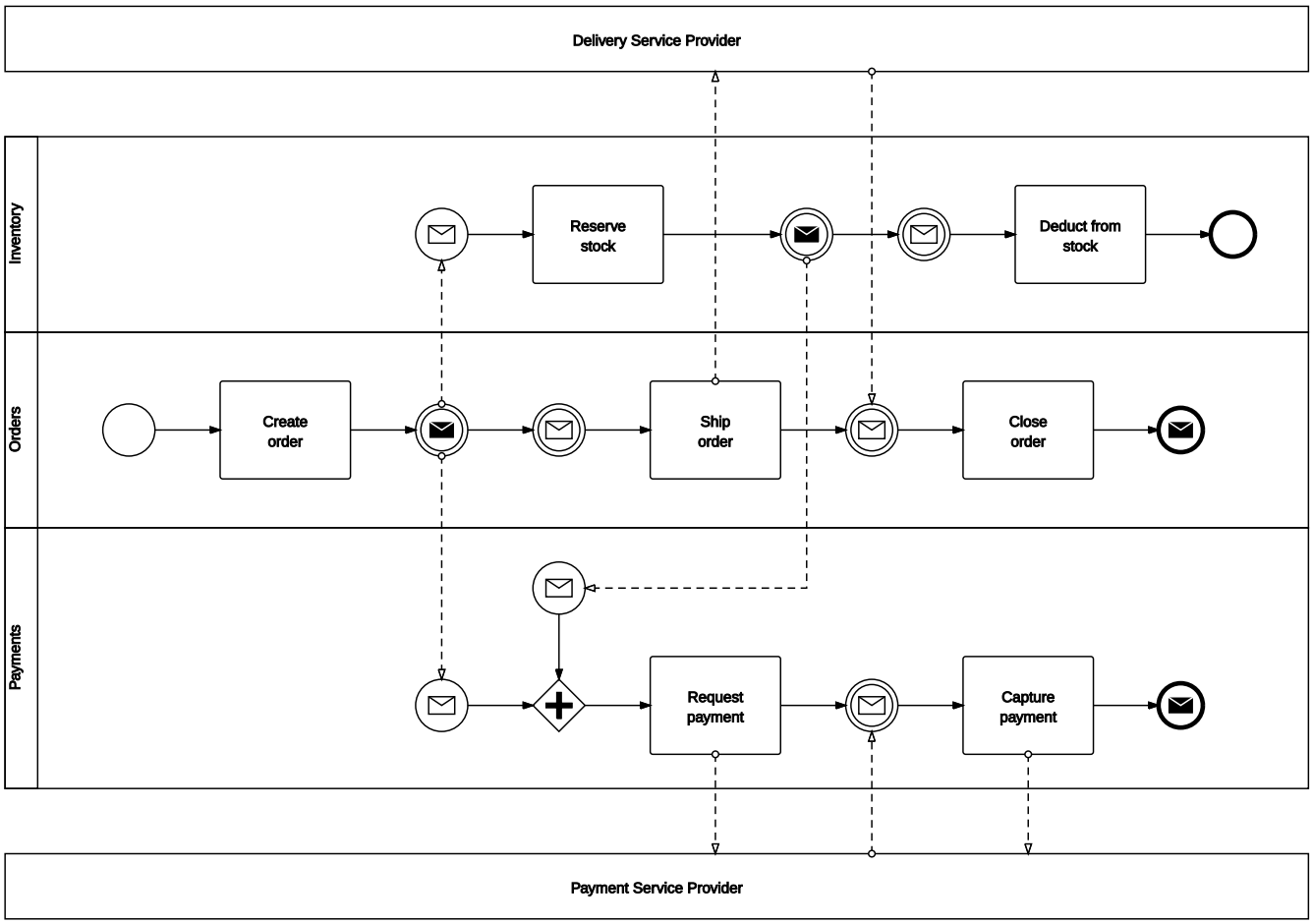
Flows



Pools and Lanes



Example


Delivery Service Provider
Inventory
Reserve stock
Deduct from stock
Orders
Create order
Ship order
Close order
Payments
Request payment
Capture payment
Payment Service Provider